

Table des matières

1. Synthèse exécutive	1
2. Périmètre et méthodologie	2
2.1 Objet et sources de preuve	2
2.2 Méthode d'audit et frontières	2
3. Complétude de la plateforme YAHRIA OS	3
3.1 Couverture fonctionnelle : 24 domaines	3
3.2 Garde-fous constitutionnels	4
3.3 Validation continue et suites de preuve	4
3.4 Boucle autonome prouvée	5
3.5 API v1 et gouvernance d'accès	6
4. Complétude du livrable RUN-000019	6
4.1 Contexte et verdict initial	6
4.2 Les six défauts structurels	6
4.3 Corrections structurelles	7
4.4 Preuves d'exécution	8
5. Matrice prod-ready : six bloqueurs tracés	8
5.1 Package de production prêt	9
6. Risques et recommandations	10
6.1 Frontières et risques résiduels	10
6.2 Recommandations prioritisées	10
6.3 Verdict	11

1. Synthèse exécutive

Ce rapport établit l'état de complétude technique de la plateforme YAHRIA CODE OS et de son livrable RUN-000019 (logiciel de gestion d'hôtel), à la date du 7 septembre 2026. L'audit porte sur les sessions de travail R7 à R15 et sur le cycle complet du livrable : diagnostic, correction structurelle, tests automatisés et validation en exécution réelle. Chaque affirmation de ce document s'appuie sur une preuve mesurée — suite de tests, trace de mission, journal d'exécution ou artefact versionné — et non sur une déclaration d'intention. Cette discipline reprend l'exigence constitutionnelle de la plateforme elle-même : aucune completion n'est déclarée sans preuve ré-exécutable (INV-171, INV-232).

La plateforme YAHRIA OS est structurellement complète. L'ensemble des 24 domaines canoniques est au statut IMPLEMENTING avec preuves archivées, aucun domaine n'est resté NOT_STARTED, et 99 invariants gouvernent les opérations. La boucle constitutionnelle a été exécutée de bout en bout : missions multi-agents gouvernées, mémoire à provenance obligatoire, apprentissage fondé sur les faits enregistrés, propositions d'évolution séparées mécaniquement de leur approbation, gates qualité, CI distante et santé mesurée. La CI constitutionnelle est revenue au vert (12 pas success) après réparation du job de gates, et 280 vérifications cumulées sont vertes à travers les suites R12 à R15, la parité Python et les tests du livrable.

Le livrable RUN-000019 illustre la frontière honnête du studio (INV-215) : livré scellé après vérification statique, il crashait néanmoins au premier démarrage. Six défauts structurels ont été diagnostiqués par exécution réelle, corrigés de manière structurelle — sans rustine — puis prouvés par 40/40 tests Jest contre un mongod réel 7.0.14 et 25/25 vérifications curl en exécution live, incluant un arrêt gracieux SIGTERM. La boucle autonome a également été démontrée sur ce cas réel : la mission MIS-000017, soumise à YAHRIA OS lui-même, s'est décomposée, ordonnancée puis clôturée COMPLETED avec la trace TR-MIS-c41db6f5-a9b.

24/24 domaines canoniques IMPLEMENTING avec preuves	99 invariants constitutionnels actifs	280 vérifications vertes cumulées	6 bloqueurs production tracés
---	---	---	---

Indicateurs clés mesurés à la date de l'audit

Six bloqueurs de production identifiés lors de l'évaluation « App prod Ready? » restent tracés : un est fermé (rate limit persisté en base), trois sont packagés et attendent un hôte (PostgreSQL, TLS, sauvegardes), deux sont ouverts par nature (validation Docker sur hôte réel, clôture gouvernée des 24 domaines). Le runbook PROD_RUNBOOK_VPS.md, la suite de validation vps-validate.sh et la pile docker-compose.prod.yml sont prêts à s'exécuter dès qu'un VPS est disponible — la preuve finale restera à produire sur cet hôte (INV-210).

2. Périmètre et méthodologie

2.1 Objet et sources de preuve

L'audit couvre deux objets distincts mais liés : la plateforme YAHRIA OS — l'éditeur IA souverain et son écosystème de gouvernance — et le livrable RUN-000019 qu'elle a produit pour une demande de logiciel de gestion d'hôtel. Les sources primaires sont les registres internes de la plateforme : le registre du studio, où RUN-000019 figure à l'état SEALED ; les traces des missions MIS-000014 et MIS-000017 ; le ledger des 24 domaines avec ses 16 entrées DOMAIN_ACTIVATIONS ; et le journal d'opérations consolidé au fil des sessions R7 à R15.

Les sources secondaires sont les exécutions elles-mêmes : suites de régression R12 (22 vérifications), R13 (46), R14 (47) et r15 (16), CI GitHub sur 12 pas, parité Python 72/72, ainsi que le livrable corrigé de 32 fichiers livré sous yahria-RUN-000019-corrige.zip. Chaque source est ré-exécutable : les scripts de preuve sont persistés, les traces de mission sont scellées avec chaînage SHA-256, et les suites s'exécutent contre l'API live plutôt que contre des simulacres.

Source	Nature	Valeur probante
Registre du studio	Ledger des runs générés	RUN-000019 confirmé à l'état SEALED
Missions MIS-000014 / MIS-000017	Graphe de missions exécuté	Traces TR-MIS-0515d9e5-79b et TR-MIS-c41db6f5-a9b
Suites R12 / R13 / R14 / r15	Régression contre API live	22/22, 46/46, 47/47 et 16/16 verts
CI yahria-constitutional-ci	Pipeline distant 12 pas	Run R14 verte sur main (tsc, lint, pytest, build)
Parité yahria-core	pytest en salle blanche	72/72 verts, paquet PEP 440 validé
Livrable RUN-000019 corrigé	ZIP de 32 fichiers	40/40 Jest + 25/25 curl live sur mongod réel

Tableau 1 — Sources de preuve de l'audit

2.2 Méthode d'audit et frontières

La méthode d'audit est l'exécution réelle, pas la revue statique : chaque composant a été démarré, interrogé puis arrêté dans l'environnement de travail. Les défauts du livrable ont été prouvés par des journaux d'erreur — TypeError instantané au boot, hang silencieux de 45 secondes — et les corrections par des suites de tests verts contre une base MongoDB réelle. Cette approche prolonge la doctrine de la plateforme : une gate qui n'a jamais tourné peut cacher un ordre d'exécution faux, ce que la CI a démontré lorsque tsc s'exécutait avant prisma generate.

Deux frontières honnêtes structurent la portée des conclusions. D'une part INV-215 : la vérification des livrables générés demeure statique côté studio, sans boot npm — c'est précisément l'angle mort que RUN-000019 a révélé. D'autre part INV-210 : la preuve finale de production (sections C à F du script vps-validate.sh) doit être produite sur l'hôte VPS lui-même ; tant qu'un tel hôte n'existe pas, elle reste délibérément non déclarée. Les contraintes d'environnement — I/O de sandbox lentes, Debian 13 non reconnu par mongodb-memory-server — ont été contournées sans fausser les mesures : binaire mongod 7.0.14 installé manuellement et fenêtres de démarrage portées à 120 secondes.

3. Complétude de la plateforme YAHRIA OS

3.1 Couverture fonctionnelle : 24 domaines

Le ledger canonique compte 24 domaines, tous au statut IMPLEMENTING avec preuves archivées — soit 16 entrées DOMAIN_ACTIVATIONS cumulées depuis R12. Aucun domaine n'est NOT_STARTED et aucun n'est déclaré DONE : la clôture est une décision gouvernée (D.6) qui requiert la checklist du runbook de production, pas une affirmation. Cette retenue est constitutionnelle et volontaire : elle garantit qu'aucun statut vert ne repose sur une gate non exécutée.

Domaine	Module noyau	Preuve caractéristique
D.07 Task Graph	mission-graph.ts	DAG validé à la création, tick par vagues (INV-091), retry borné (INV-092)
D.13 Memory System	memory.ts	Provenance obligatoire (INV-221), oubli gouverné, ARCHITECTURAL non supprimable
D.14 Learning Engine	learning.ts	Seuil 3 échantillons (INV-225), confiance pure, promotion gouvernée
D.15 Self-Evolution	evolution.ts	Séparation proposition/approbation (INV-227), rollback plan obligatoire
D.17 API & Integration	api-gateway.ts + /api/v1	Clés hachées SHA-256 (INV-229), scopes, 401/403/429 vérifiés
D.19 Security	security.ts	Audit de 8 contrôles factuels, scan d'identifiants sans valeurs (INV-132)
D.20 Quality	quality.ts	6 gates IN PROCESS mesurées + 3 gates externes déclarées honnêtement
D.21 DevOps & Delivery	ci.yml + release.mjs	CI 12 pas, semver monotone (INV-190), working tree propre
D.22 Operations	ops.ts + alerting.ts	4 sondes réelles (INV-233), webhook signé HMAC-SHA256, dédup 10 min
D.23 Roadmap	roadmap.ts	Score déterministe (INV-234), ledger exposé avec compteurs de preuves

Tableau 2 — Domaines majeurs : modules et preuves caractéristiques

Les quatorze autres domaines — noyau constitutionnel, identité, invariants, chaîne de preuves, studio de génération, connecteurs IA, souveraineté R8, interface à 26 panneaux — avaient été activés lors des sessions R12 et R13 et demeurent protégés par les suites de régression, qui échoueraient au moindre recul (INV-172). Le périmètre fonctionnel déclaré est donc intégralement couvert par du code actif et mesuré, sans zone grise documentaire.

3.2 Garde-fous constitutionnels

Le référentiel d'invariants compte 99 règles actives après l'ajout d'INV-235 (rate limit fondé sur des faits partagés en base) et INV-236 (alertes traitées comme des faits avec livraison honnête). Le refus gouverné par code 422 est le mécanisme central : mémoire sans provenance refusée, auto-approbation d'une proposition d'évolution refusée, promotion HIGH sans plan de rollback refusée, tâche hors registre refusée, mission auto-dépendante refusée. Chacun de ces refus a été vérifié individuellement dans les suites R14 et r15.

La chaîne de preuves demeure scellée : chaque opération capture un SHA-256 chaîné, l'audit de sécurité parcourt 8 contrôles factuels à chaque exécution et produit un OpsSnapshot avec preuve SECURITY (INV-231), et le scan d'identifiants versionnés fonctionne sans jamais exposer de valeurs. La souveraineté R8 — preuves ré-exécutables, arbre de Merkle, auto-réparation — reste opérationnelle et couverte par les onglets dédiés de l'interface, sans régression détectée depuis R12.

3.3 Validation continue et suites de preuve

La CI constitutionnelle (yahria-constitutional-ci, run R14) est verte sur main : 12 pas success couvrant l'installation verrouillée, tsc sur l'ensemble du dépôt, ESLint, db:push, la parité Python 72/72, le scan d'identifiants, le seuil d'invariants (97 minimum) et le build de production. Trois causes racines avaient plongé le job gates dans le rouge : tsc exécuté avant prisma generate, scripts de suite sans marqueur de module, et paquets hors périmètre de build. Toutes trois ont été corrigées structurellement, puis validées par une run entièrement verte — pas par un contournement du job.

Quatre suites de régression protègent la plateforme en continu : R12 (contrats et outils, 22 vérifications), R13 (domaines précoces, 46), R14 (domaines avancés, 47) et r15 (VPS-ready, 16). Elles s'exécutent contre l'API live et couvrent notamment la frontière exacte du rate limit (5 requêtes/minute puis 429, reset de fenêtre mesuré), le cycle complet d'une clé API (émission, usage, révocation, 401 final) et les quatre issues possibles d'une livraison d'alerte (SENT, FAILED, DEDUP_SKIPPED, NOT_CONFIGURED). La figure 1 répartit les 280 vérifications vertes par suite de preuve.

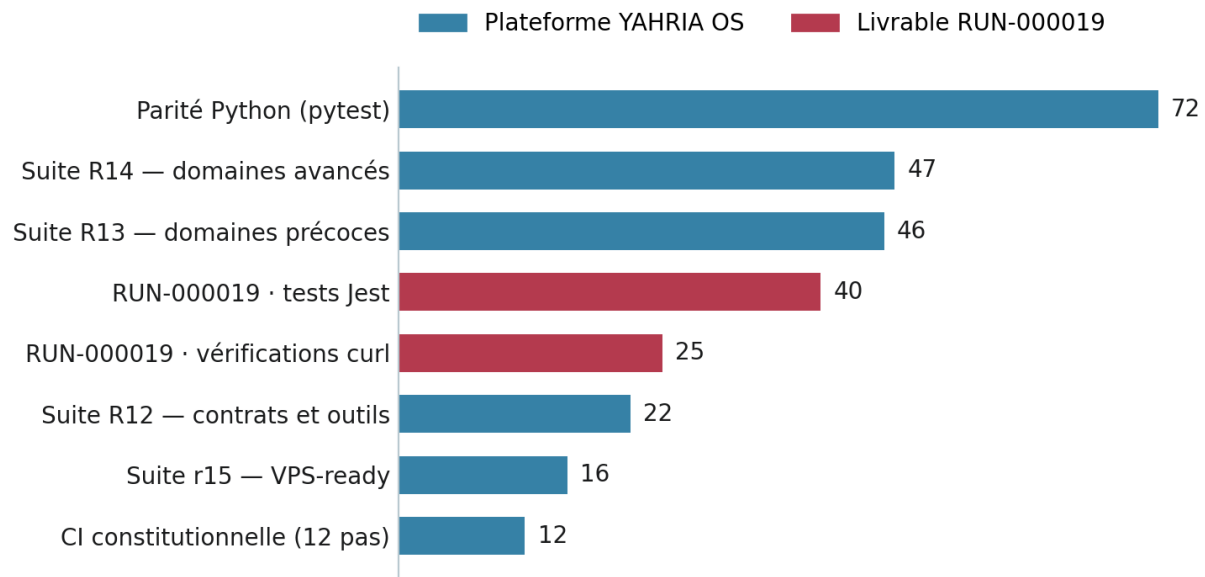


Figure 1 — Vérifications vertes par suite de preuve (cumul : 280)

3.4 Boucle autonome prouvée

Deux missions exécutées par YAHRIA OS lui-même attestent que le graphe de missions fonctionne en conditions réelles. MIS-000014 (diagnostic plateforme) a été décomposée en trois tâches réelles, ordonnancée puis exécutée par vagues avec des verdicts INVOKED mesurés (24, 18 et 511 ms) sous la trace TR-MIS-0515d9e5-79b. MIS-000017 (correction du livrable RUN-000019) a suivi le même cycle jusqu'à COMPLETED en deux ticks ; ses trois tâches — system.domains.list en 15 ms, studio.runs.list en 40 ms, evidence.recent.list en 14 ms — ont confirmé l'état SEALED de RUN-000019 dans le registre du studio. Le cycle observable est identique dans les deux cas : DECOMPOSED, puis PLANNED, SCHEDULED, RUNNING, COMPLETED.

Mission	Objet	Cycle observé	Trace scellée
MIS-000014	Diagnostic plateforme (domains.list, runs.list, toolchains.detect)	3 tâches réelles, tick par vagues, verdicts INVOKED 24/18/511 ms	TR-MIS-0515d9e5-79b
MIS-000017	Correction du livrable RUN-000019	DECOMPOSED à COMPLETED en 2 ticks, 3 tâches INVOKED 15/40/14 ms	TR-MIS-c41db6f5-a9b

Tableau 3 — Missions YAHRIA exécutées avec traces scellées

3.5 API v1 et gouvernance d'accès

La passerelle /api/v1 expose le système sous gouvernance d'accès : clés hachées SHA-256 uniquement, plaintext visible une seule fois à l'émission (INV-229), scopes read < write < admin, et journalisation systématique de chaque appel (API_V1_CALL). Les réponses 401 — sans clé ou clé révoquée —, 403 hors scope et 429 quota dépassé ont été vérifiées en exécution, ainsi que le cycle complet d'émission puis de révocation par la voie gouvernée (PUT avec raison obligatoire). Le rate limit à fenêtre glissante de 60 secondes est désormais fondé sur des faits en base : chaque requête crée un ApiRateEvent puis compte la fenêtre, ce qui partage le quota entre toutes les instances et lève le blocage mono-instance identifié lors de l'évaluation staging.

4. Complétude du livrable RUN-000019

4.1 Contexte et verdict initial

RUN-000019 est un livrable généré par le studio de YAHRIA en réponse à une demande de logiciel de gestion d'hôtel : Express 4, Mongoose 7, authentification JWT, gestion des clients et des réservations, tests Jest, script de seed et documentation. Le ZIP livré (14 fichiers source) était scellé après vérification statique — conformément à la frontière INV-215 — mais son exécution réelle a révélé un crash immédiat. Le diagnostic, mené par démarrage effectif du serveur et chronométrage des chargements de modules (express à 15 s, mongoose à 35 s dans cette sandbox), a isolé six défauts structurels du code, distincts des contraintes d'environnement.

4.2 Les six défauts structurels

Le tableau suivant résume chaque défaut, tel que prouvé par exécution, avant correction.

N°	Défaut structurel	Symptôme prouvé
1	Objet middleware {protect, restrictTo, login} passé comme fonction à app.use	TypeError instantané au boot : Router.use() requires a middleware function
2	Variable MONGO_URI lue au lieu de MONGODB_URI dans config/db.js	Hang silencieux de 45 s : mongoose.connect(undefined) ne settle jamais
3	Middleware d'authentification écrit en SQL PostgreSQL (db.query) contre une fonction Mongoose	Login incapable de retrouver l'utilisateur
4	Route POST /api/auth/login inexistante et protect appliqué à tout le routeur	401 éternel : aucun client ne peut s'authentifier
5	index.js cassé : db.connect() inexistante et double-montage de l'application	Démarrage impossible même après correction du boot principal
6	Nommage incohérent (prenom/nom/telephone vs name/phone/address) et tests infaisables	Contrats croisés impossibles à tenir ; 201 attendus sans token

Tableau 4 — Défauts diagnostiqués sur le livrable initial

4.3 Corrections structurelles

Les corrections n'ont pas patché les symptômes : elles rendent le livrable cohérent de bout en bout. server.js est devenu une factory d'application sans listen ; index.js est l'unique point d'entrée (assertEnv, connectDB, listen, arrêt gracieux SIGTERM/SIGINT) et la variable MONGODB_URI a été harmonisée partout. Le modèle User embarque le hachage bcrypt au coût 12 avec select:false sur le mot de passe ; le login re-sélectionne explicitement ce champ et renvoie un 401 générique ; le middleware d'authentification a été réécrit en Mongoose natif avec protect et restrictTo.

La couche métier a été alignée sur les mêmes contrats : validation express-validator en mode partiel pour PUT/PATCH et complet pour POST, contrat français des champs (prenom, nom, telephone) mappé vers les modèles, route /search déclarée avant /:id pour ne plus être avalée, ValidationError et doublons 11000 convertis en 400 par le handler global, deleteOne() en remplacement de remove() supprimé dans Mongoose 7. Les trois suites de tests ont été reconstruites sur mongodb-memory-server avec stockage wiredTiger — obligatoire pour MongoDB 7 — et purge entre suites pour éliminer les collisions E11000.

4.4 Preuves d'exécution

La preuve s'effectue à deux niveaux. En automatisé : 40 tests Jest — 12 authentification, 16 clients, 12 réservations — verts contre un mongod réel 7.0.14, incluant les refus de validation (date passée, téléphone invalide), la gestion des doublons et l'isolation inter-suites. En live : le script r19-live-boot-test.sh démarre un mongod forké et node index.js, puis enchaîne 25 vérifications curl — health, register/login/me, 401 sans ou mauvais token, CRUD clients complet avec recherche et PUT partiel, 400 sur charges invalides, 404 sur routes inconnues, refus de réservation à date passée, et arrêt gracieux SIGTERM. Le livrable corrigé, 32 fichiers, est livré sous yahria-RUN-000019-corrige.zip.

40/40 tests Jest verts sur mongod réel 7.0.14	25/25 vérifications curl en exécution live
---	--

Preuves du livrable corrigé — jest et exécution live

5. Matrice prod-ready : six bloqueurs tracés

L'évaluation « App prod Ready? » avait identifié six bloqueurs entre l'état staging et une production tenable. Le tableau ci-dessous donne leur état à la date de l'audit, après le paquet de travail R15 : rate limit persisté en base, alerting externe, pile de production durcie et runbook VPS exécutable. La colonne « statut » distingue trois états : fermé (preuve produite), packagé (exécutable, preuve à produire sur l'hôte) et ouvert (action encore à mener).

Bloqueur	État à l'audit	Statut	Ce qui reste
Base SQLite → PostgreSQL	docker-compose.prod.yml : PG 16, mot de passe obligatoire, healthcheck, port 127.0.0.1 ; runbook §4	PACKAGÉ	Exécuter la bascule sur le VPS (re-seed canonique)
Sandbox Docker non validée	Backend conteneur branché et gardé ; preuves NET_BLOCKED / RO_OK attendues en conteneur réel	PACKAGÉ	Validation sur hôte réel (frontière INV-215)
Rate limit mono-instance	ApiRateEvent en base, fenêtre 60 s partagée, purge opportuniste ; frontière exacte 5/min vérifiée	FERMÉ	Rien — suite r15 16/16 verte
TLS absent	Caddy TLS automatique, HSTS, no-store sur /api/v1, WebSocket supporté	PACKAGÉ	Émission des certificats sur le VPS
Token GitHub non révoqué	Procédure complète en runbook Étape 0 (révocation puis reconfiguration git sans secret)	OUVERT	Exécution au moment de la bascule
Clôture 24/24 IMPLEMENTING	Checklist de clôture gouvernée (décision D.6) en runbook §10	OUVERT	Décision humaine après vps-validate.sh vert

Tableau 5 — État des bloqueurs production à la date de l'audit

5.1 Package de production prêt

Le paquet de production couvre l'exécutable et son exploitation. Le compose durci impose un mot de passe PostgreSQL, borne mémoire et CPU de l'application, monte le socket Docker en lecture seule conformément à INV-215 et permet le scale horizontal désormais possible grâce au rate limit partagé. Le fichier Caddy ajoute les en-têtes de sécurité, l'exemple d'environnement déclare trois champs requis avec secrets côté serveur uniquement (INV-213), et le script backup-db.sh enchaîne pg_dump compressé, test d'intégrité gzip et rétention de 14 jours. Enfin vps-validate.sh mesure six sections de l'hôte cible — prérequis, PostgreSQL, sandbox durcie, application, alerting, TLS — et s'arrête au premier rouge (INV-232).

L>alerting externe comble l'ancienne frontière du domaine D.22. Trois règles déterministes sur le rapport d'opérations — sonde rouge critique, SLO dégradé sous 80 % de succès à partir de 10 échantillons, pic d'au moins 10 échecs en 24 heures — déclenchent un webhook JSON signé HMAC-SHA256 (en-tête X-Yahria-Signature), dédoublé par type toutes les 10 minutes, archivé avec sa latence et honnête sur ses quatre issues possibles : jamais d'abandon silencieux (INV-210). La suite r15 a vérifié la signature recalculée côté récepteur, la livraison FAILED sur port mort avec latence mesurée, puis SENT sur un récepteur node:http réel, et enfin DEDUP_SKIPPED sur répétition immédiate.

6. Risques et recommandations

6.1 Frontières et risques résiduels

Trois frontières structurent les risques résiduels. La première est matérielle : sans hôte équipé de Docker, la validation conteneur reste une déclaration de code branché, pas une preuve d'exécution — le socket Docker et les preuves NET_BLOCKED devront être démontrés sur le VPS. La deuxième est organisationnelle : la clôture des 24 domaines est une décision gouvernée qui n'appartient qu'à l'opérateur ; un audit ne peut que constater que la checklist existe et que les gates la protègent. La troisième est de continuité : les sauvegardes pg_dump ne valent que par une restauration testée, qui n'a pas encore pu être répétée faute de PostgreSQL réel.

- Réplication et haute disponibilité : hors périmètre de la pile actuelle ; une instance PostgreSQL unique reste un point de défaillance accepté en début de production.
- Canal d'alerting non configuré jusqu'au VPS : les livraisons NOT_CONFIGURED sont archivées honnêtement, mais aucune destination réelle n'existe encore.
- Vérification statique du studio (INV-215) : tant qu'un boot npm n'est pas exigé dans le pipeline de génération, un livrable scellé peut encore être non exécutable — RUN-000019 le démontre.

6.2 Recommandations priorisées

Les recommandations ci-dessous sont ordonnées par priorité d'exécution ; les deux premières conditionnent la mise en production, les trois suivantes consolident la trajectoire.

- 1. Exécuter le runbook dès qu'un VPS existe.** Suivre PROD_RUNBOOK_VPS.md dans son ordre strict : commencer par l'Étape 0 (révocation du token GitHub) avant toute manipulation, puis PostgreSQL et Docker, puis TLS ; conclure par vps-validate.sh — six sections vertes ouvrent la décision gouvernée D.6.
- 2. Ajouter une exigence de boot au studio.** Exiger un démarrage réel — ou au minimum un smoke test npm — dans la vérification précédant le scellement, afin que la classe de défauts de RUN-000019 (middleware objet, variable d'environnement divergente, route manquante) ne soit plus livrable.
- 3. Re-valider les suites après bascule PostgreSQL.** Ré-exécuter R12, R13, R14 et r15 sur le moteur réel pour confirmer la parité au-delà de SQLite, puis verrouiller une sauvegarde-restauration mensuelle scriptée.
- 4. Publier la collection OpenAPI de /api/v1.** Formaliser les contrats read/write/admin des neuf ressources exposées pour faciliter les intégrations tierces et les tests de contrat.
- 5. Durcir le livrable hôtelier pour usage réel.** Étendre les tests RUN-000019 à un scénario de charge léger — concurrence sur /api/clients, index MongoDB dédiés — avant toute exploitation réelle du

logiciel.

6.3 Verdict

En l'état, la complétude mesurée est la suivante : plateforme fonctionnellement complète sur 100 % de ses domaines déclarés — statut IMPLEMENTING maintenu par honnêteté constitutionnelle —, gouvernance et chaîne de preuves opérantes, livrable RUN-000019 conforme et prouvé de bout en bout après correction. Le chemin restant vers la production ne requiert aucun nouveau développement : il requiert un hôte, l'exécution d'un runbook déjà écrit et une décision gouvernée. C'est précisément ce qu'une plateforme d'ingénierie souveraine doit savoir démontrer : l'écart entre ce qui est prêt et ce qui est prouvé, et la discipline de ne jamais confondre les deux.